Національний технічний університет України "Київський політехнічний інститут імені Ігоря Сікорського"

КРИПТОГРАФІЯ

ЛАБОРАТОРНА РОБОТА №4

Криптоаналіз шифру Віженера

Валюх Артем ФБ-43

Журавський Владислав ФБ-43

Варіант 3

Мета роботи

Ознайомитися з перевіркою чисел на простоту, генерацією параметрів RSA, шифруванням, розшифруванням, цифровим підписом і протоколом передавання ключа з підтвердженням справжності відправника.

Постановка задачі та параметри роботи

Побудувати RSA-криптосистему для двох абонентів A і B: самостійно перевіряти простоту чисел, отримати по два 256-бітних простих числа для кожного абонента, згенерувати RSA-ключі, перевірити шифрування і цифрові підписи та реалізувати передавання секретного ключа з автентифікацією відправника.

Параметри: Python 3; генератор випадкових чисел SystemRandom; 256 біт на просте число; відкритий показник e = 65537; 24 раунди тесту Міллера–Рабіна; умова nA ≤ nB. Готові бібліотеки тестів простоти не використовувалися.

Порядок виконання роботи

1. Перевірка простоти та генерація простих чисел

Спочатку перевірялися малі прості дільники, потім виконувався тест Міллера–Рабіна. Для одного числа зафіксовано результат однієї основи тесту.

Параметр	Отримане значення
Згенероване просте число (256 біт)	72496873660510562850961417455462256046749306777882248243567933963360940861417
Основа a	35376025270230636711957859937274486865100285530043986200014445138397803925957
s у розкладі n − 1 = d · 2^s	s: 3
d у розкладі	d: 90621092075638203563701771819327820058436633472352810304459917454201176076 77
a^d mod n	14081061170418812836168909004744329983652278305512901159087307804494414011619
Результат перевірки	Раунд успішно пройдено

У перевіреному раунді послідовне піднесення до квадрата дало значення n − 1, отже кандидат пройшов перевірку для обраної основи. Сам по собі один раунд не є математичним доведенням простоти.

Кандидат, який не пройшов перевірку	Причина
597104782400005752640341383370075877713863050071753852681826502091568686313753	Міллер–Рабін
829147029064851962299644395937487612034430530673734376422863526135031112967 09	Ділиться на 17
626794014747882405206749106747346253068828607921906003783386951643714874772 57	Ділиться на 3
874938935293500324648574947727820505180032269367645194779742238952383141741 65	Ділиться на 5

2. Генерація двох пар простих чисел

Параметр	Значення (десяткова система)
p	106160493736384039663913188848294292761907914422623475007442448093773522615177
q	582356521840661978607535265116709808904242687172972131045305188016971691354 71
p_1	794484933700008963437661847863819357018994194709117544793008381165100425169 49
q_1	992684537816520484309305395044777960050422471859411994671461074144085301554 39

З отриманих чисел обчислено модулі nA = p·q та nB = p_1·q_1. Умова nA ≤ nB виконана (True), що необхідно для коректної роботи протоколу розсилання ключів.

3. Генерація ключових пар RSA

Параметр	Абонент A	Абонент B
Модуль n	618232558892079911729839549622592662906142405800419122649030458813641800363932610906664080676790680134422989896928583608128184529108196514247021611364 3367	788672909212182317607142985714923411881646203168843280103305165985363193408374347187155993745499543167850527775039795013738162618714529322153967376203561 1
Відкритий показник e	65537	65537
Секретний показник d	345420474067340435598387701428665876339078939731662221554839988256696550100332917957822325243866827324572883046421123963926122424259505628107817649745099 3	638404227134386254315256044557680195924765110977114240955190793834376266998997681078993331237724048994188130179732911178985838869678500381375809791103833
φ(n)	618232558892079911729839549622592662906142405800419122649030458813641800363916171292072035653038213462886993369563350389814192460296999217557474542189272 0	788672909212182317607142985714923411881646203168843280103305165985363193408356475492440828451022073495421441801869100847072477323319884627600875518936322 4
Перевірка	Результат	
p, q — різні прості числа	Пройдено	
n = p · q	True	
gcd(e, φ(n)) = 1	True	
e · d mod φ(n) = 1	True	
nA ≤ nB	True	

4. Шифрування, розшифрування та цифровий підпис

Для кожного абонента було обрано випадкове ціле повідомлення M. Отримано шифротекст C та цифровий підпис S; після розшифрування відновлено первинне M. Для підпису також проведено негативну перевірку із зміненим повідомленням.

Результати для абонента A

Показник	Значення (десяткова система)
M	587620202090709431782613987452721735901650709904

	205702342134183325351678283592616242704473125708222932551211984698251186504959531836814885957403949195854 6
C	262139989588256181083821400981941423475992598206882516368616236963803845152083248542981124023697912228408652598235846182316665004913886137768696652473754 2
S	437478400535799248808958853451713403475048804705371864562694720047662747251678481134299612405392998852868983761179851948721009372196052580892898885241922 5
Відновлене M	Збігається з вихідним
Перевірка підпису	True
Підпис після зміни повідомлення	False

Результати для абонента B

Показник	Значення (десяткова система)
M	429411672064925869205638898676242932460416928975582235584690420291111744199881142185038313497611932125246625819109394611475839480529659743026614505221317 8
C	394852931168026391108799195681900940419545788460109245302415475321126211156402838439715547395147802975823455077479115943201861346426986537677543310246491 0
S	854856010939029911783456482345159993224498533503540912374519806374288746195924376871873570486664578666933231510599524672461162623620289083860148950349293
Відновлене M	Збігається з вихідним
Перевірка підпису	True
Підпис після зміни повідомлення	False

5. Конфіденційне передавання ключа від A до B

Абонент A обрав секретне значення k, сформував підпис секретним ключем A, а потім зашифрував k і підпис відкритим ключем B. Абонент B відновив обидва значення своїм секретним ключем та перевірив підпис відкритим ключем A.

Крок	Учасник	Дія
1	A	Обирає $0 < k < nA$
2	A	Шифрує k відкритим ключем B → k_1
3	A	Підписує k секретним ключем A → S
4	A	Шифрує підпис відкритим ключем B → S_1
5	A → B	Передає пару (k_1, S_1)
6	B	Розшифровує k_1 і S_1 секретним ключем B
7	B	Перевіряє S відкритим ключем A

Параметр	Отримане значення
Початковий k	437768049508970124051053164247943031887998535406

	90803489357422559604863187115305868493822621918317923733999129451966530640646333577104415303743208803279 35
Переданий k_1	51294759041633770948585438372436775123146865192756571170205208857430123105412290485292382510583159943136070259805915460598443971774664549538615118301998 98
Переданий S_1	52561541432436421705727605790084813728740472330070601689791042087043917919566846458896796975945480269214180605090380176710907873539985761276041284477959 61
Відновлений k	43776804950897012405105316424794303188799853540690803489357422559604863187115305868493822621918317923733999129451966530640646333577104415303743208803279 35
Автентичність	True

Відновлений ключ збігається з відправленим. Конфіденційність забезпечується відкритим ключем одержувача B, а перевірка справжності — підписом, сформованим секретним ключем A та перевіреним відкритим ключем A.

6. Взаємодія із зовнішнім тестовим середовищем

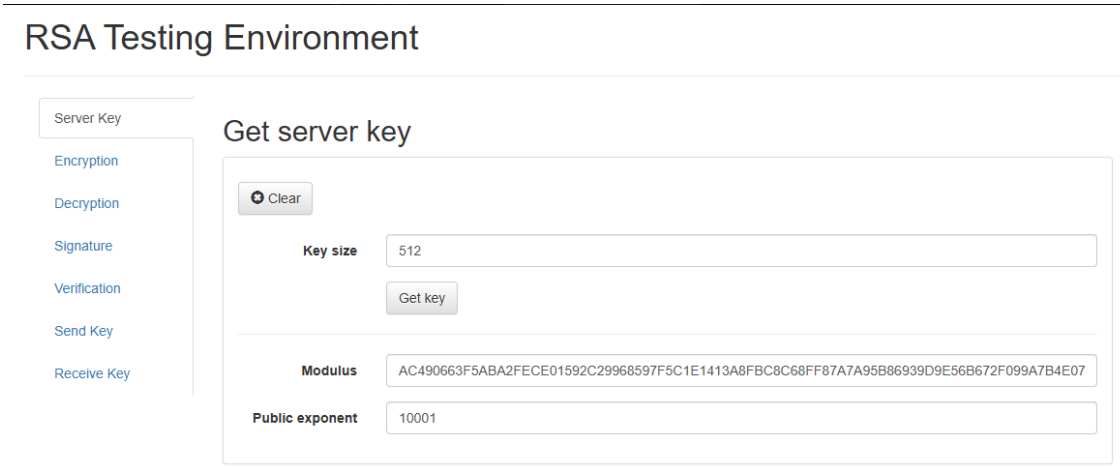
Методичка рекомендує перевіряти сумісність із RSA Testing Environment (http://asymcryptwebservice.appspot.com/?section=rsa) у двох напрямках: шифрування/розшифрування, підпис/перевірка та передавання/одержання ключа.

Результати взаємодії з RSA Testing Environment підтверджені наведеними нижче скріншотами. П’ять перевірок завершилися успішно: два напрями шифрування, два напрями перевірки цифрового підпису та Python SendKey → Server ReceiveKey. У напрямі Server SendKey → Python ReceiveKey отримано Verification: False; для цієї операції потрібна повторна перевірка із правильним відкритим модулем одержувача.

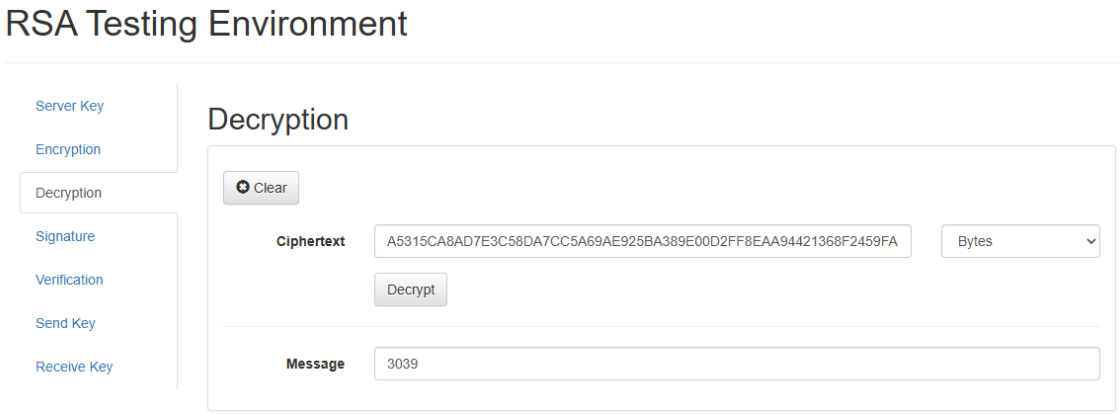
Перевірка	Статус
Отримання відкритого ключа сервера	Підтверджено скріншотом (512 біт)
Python Encrypt → Server Decrypt	Успішно: сервер повернув 3039 (HEX)
Server Encrypt → Python Decrypt	Успішно: Python отримав 3039 (HEX), True
Python Sign → Server Verify	Успішно: Verification = true
Server Sign → Python Verify	Успішно: Signature valid = True
Python SendKey → Server ReceiveKey	Успішно: Key = 3039, Verification = true
Server SendKey → Python ReceiveKey	Невдала спроба: Verification = False; перевірити модуль B

Нижче наведено скріншоти проведених перевірок та пояснення їхніх результатів.

6.1. Шифрування Python → розшифрування сервером



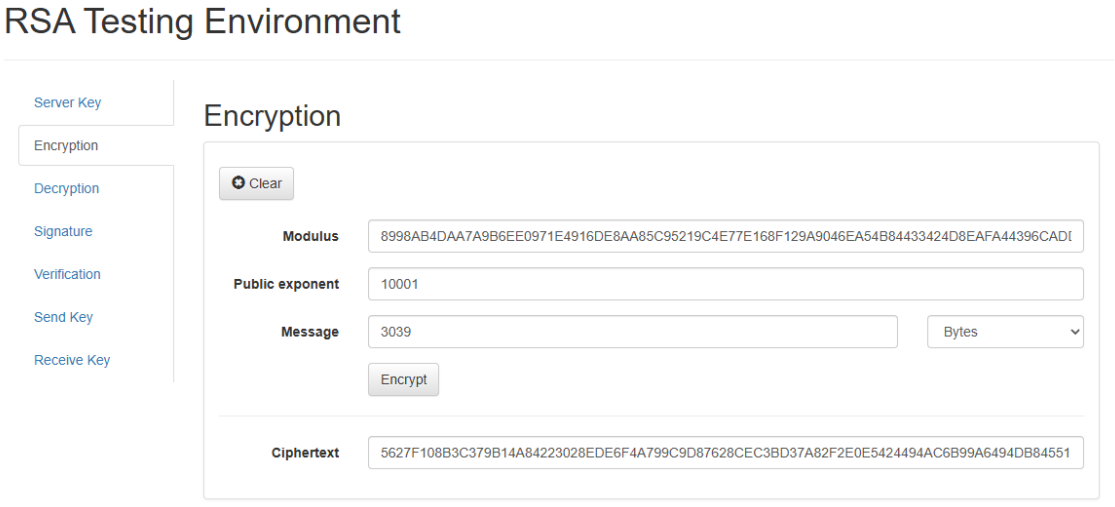
Відкритий ключ RSA тестового сервера (512 біт).



Сервер розшифрував повідомлення до 3039 (HEX).

Власною реалізацією зашифровано M = 12345 (3039 HEX) відкритим ключем сервера. Сервер відновив 3039 (HEX), отже перевірка успішна.

6.2. Шифрування сервером → розшифрування Python



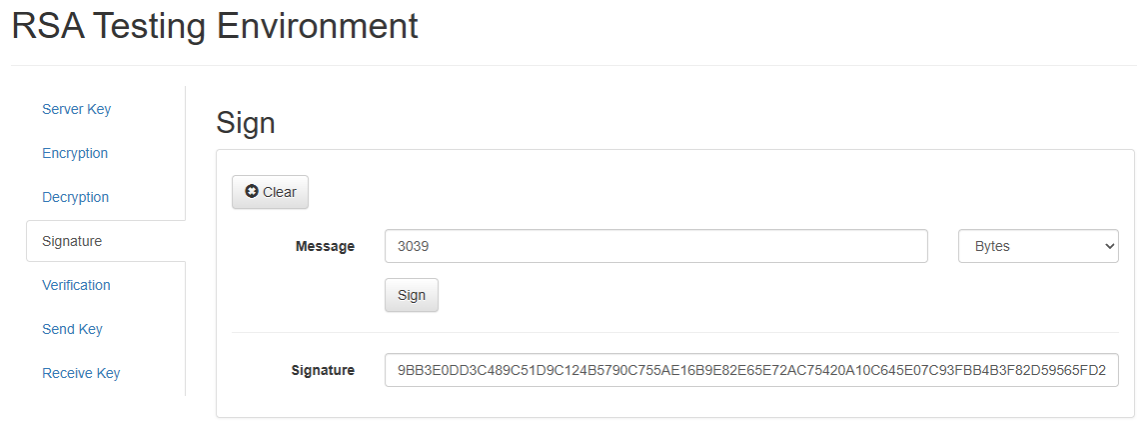
Сервер зашифрував 3039 (HEX) відкритим ключем абонента A.

```
1 — Server Encryption → Python Decryption
2 — Server Signature → Python Verification
3 — Python Signature → Server Verification
4 — Python SendKey → Server ReceiveKey (сервер 1024 біт)
5 — Server SendKey → Python ReceiveKey (сервер 256 біт)
0 — Вийти
Пункт: 1
На сайті Encryption: Modulus A та Exponent A наведені вище; Message = 3039; Bytes; Encrypt.
Ciphertext з сайту (HEX): 5627F108B3C379B14A84223028EDE6F4A799C9D87628CEC3BD37A82F2E0E5424494AC6B99A6494DB84551B73648E9
7651417E27CF5250DAC06A775FDABE85715
Decrypted M: 3039
УСПІХ: True
```

Python відновив 3039 (HEX); УСПІХ: True.

Сервер сформував шифротекст для відкритого ключа A. Після розшифрування приватним ключем A програма повернула початкове значення 3039 (HEX); результат True.

6.3. Підпис сервера → перевірка Python



Створення сервером цифрового підпису повідомлення 3039 (HEX).

```
1 — Server Encryption → Python Decryption
2 — Server Signature → Python Verification
3 — Python Signature → Server Verification
4 — Python SendKey → Server ReceiveKey (сервер 1024 біт)
5 — Server SendKey → Python ReceiveKey (сервер 256 біт)
0 — Вийти
Пункт: 2
На сайті Server Key згенеруй ключ (наприклад 512 біт). Потім Signature: Message=3039, Bytes, Sign.
Modulus сервера (HEX): AC490663F5ABA2FECE01592C29968597F5C1E1413A8FBC8C68FF87A7A95B86939D9E56B672F099A7B4E075E5E9996F5B
82A3C7240E434ECFD5E7A1AA5F3EB8C3
Public exponent сервера (HEX): 10001
Signature із сайту (HEX): 9BB3E0DD3C489C51D9C124B5790C755AE16B9E82E65E72AC75420A10C645E07C93FBB4B3F82D59565FD28EE8B1329
142689B0B52E9AD01E38A31615F5BC64144
Signature valid: True
```

Сервер створив підпис повідомлення 3039 у режимі Bytes. Python перевірив його відкритим ключем сервера, отримано Signature valid: True.

6.4. Підпис Python → перевірка сервером

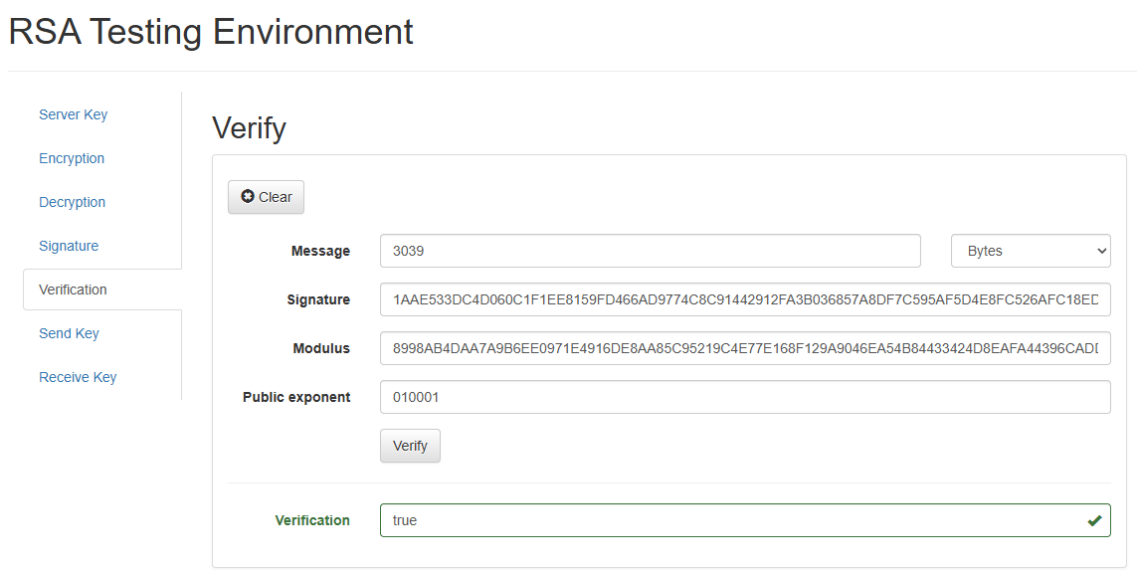
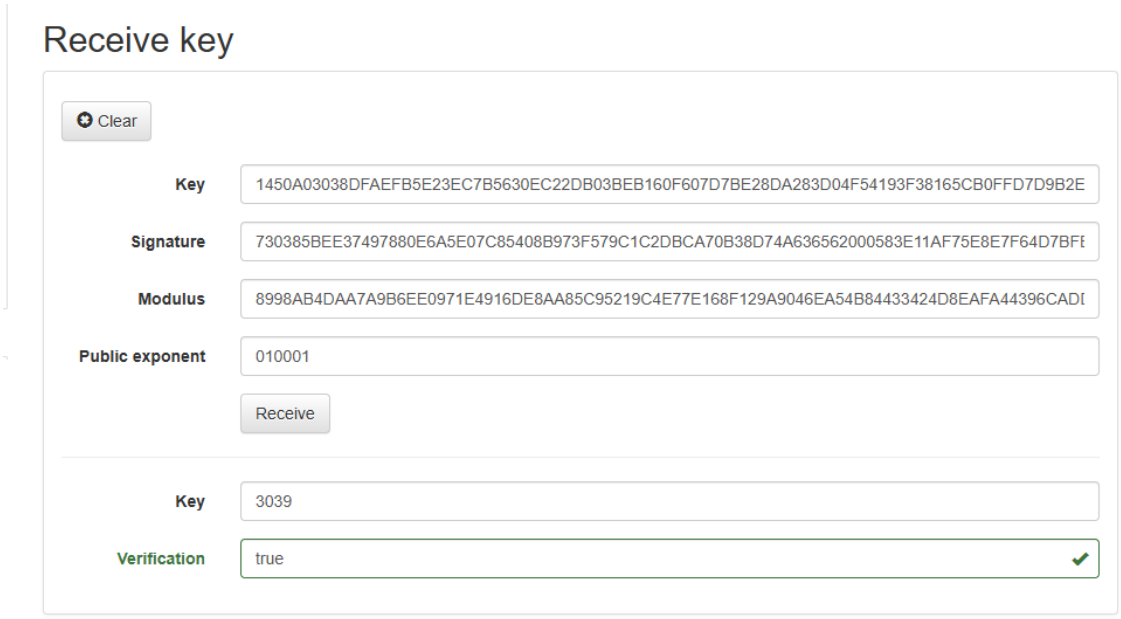


Рисунок 6.7 – Сервер підтвердив цифровий підпис Python: Verification = true.

За допомогою приватного ключа A створено підпис повідомлення 3039 (HEX), який сервер перевірив за відкритим ключем A. Результат Verification = true.

6.5. Python SendKey → Server ReceiveKey



Сервер отримав ключ 3039 (HEX); Verification = true.

```
Пункт: 4
На сайті Server Key згенеруй 1024-бітний ключ сервера, скопіюй його поля.
Modulus сервера (1024 біт) (HEX): 8E88A8C7270F6776685D20D2AA668DD3E7082135C84ED194906A2CD89784B18DA9CC2DC112C20E1C4499B
9F5EE59FF04A0F342F433A874FC321B81B631AC6B653CB69510806D45C7B902418B0609A5B7FC1F054D4B1FED6B3C1E641BAECC410B132D889D1B15
EE512AE0A622A2F5246840C92BDDCA2BBB88A1BC9D241564A835
Public exponent сервера (HEX): 10001
На сайті Receive Key введи:
Key: 1450A03038DFAEFB5E23EC7B5630EC22DB03BEB160F607D7BE28DA283D04F54193F38165CB0FFD7D9B2E8FAB100C5DE2F772FE0C0156143779
8217BB1F4A1F602D9E0EAF49D0EC99D27532F0F1F9E89FC1914724BE2B84289290629EE7152F5B552DA6549E3171FA999B643A8FF58561303C464B5
6A43BD24B179E9819560E66
Signature: 730385BEE37497880E6A5E07C85408B973F579C1C2DBCA70B38D74A636562000583E11AF75E8E7F64D7BFBA5F9A2AACAA05141AAEEA1
22FA00FAD6F5965682E9C6F9E1EDDEDFBE7A0936E74654E59EC6232023DEEADD86B4A6FB8955FA9517AB66A9DCF417EC488C85F72EF5850A156C692
9CD925EF91E3B9EA789C3C2E19F63
Modulus A: 8998AB4DAA7A9B6EE0971E4916DE8AA85C95219C4E77E168F129A9046EA54B84433424D8EAFA44396CADD74D4E0F1FDA7AE90AAB6D30
511E1D38AE8918336 7B5
Public exponent A: 010001
Натисни Receive. Очікується Key=3039 (можливо 003039 з нулями), Verification=true.
```

Дані, підготовлені Python для протоколу передавання ключа.

Python зашифрував ключ і підпис для 1024-бітного ключа сервера, використавши приватний ключ A для підпису. Сервер відновив значення 3039 (HEX) та підтвердив справжність: Verification = true.

Під час локального приймання значень із Send Key програма вивела Verification: False. На скріншоті Send Key в полі Modulus видно значення A53C7555… (модуль 256-бітного сервера), тоді як у скрипті для одержувача B показано CC877446… (512-бітний модуль). Це пояснює ймовірну причину невідповідності. Для повторного тесту слід використати саме відкритий ключ одержувача B, отримати нові Key та Signature на сервері й перевірити підпис відкритим ключем сервера. Успішне завершення цього напряму на поточних скріншотах не підтверджено.

Особливості реалізації та труднощі

Для передавання ключа перевіряється умова $nA \le nB$. У разі невідповідного порядку модулів пари простих чисел переставляються. Перевірка простоти містить пробні ділення для швидкого відхилення кандидатів, а тест Міллера–Рабіна зменшує ймовірність помилкового прийняття складеного числа. Демонстраційна реалізація RSA використовує операції без OAEP/PSS і не призначена для захисту реальних даних.

Відповіді на контрольні питання

1. Які перевірки виконуються під час генерації простих чисел?

Контроль потрібної бітової довжини та непарності кандидата; пробне ділення на малі прості дільники для швидкого відсіювання складених чисел; 24 раунди тесту Міллера–Рабіна для ймовірнісної перевірки простоти.

2. Які умови коректності параметрів RSA?

p і q — різні прості числа; $n=pq$; $\varphi(n)=(p-1)(q-1)$; $1<e<\varphi(n)$; $\gcd(e,\varphi(n))=1$; $e \cdot d \bmod \varphi(n)=1$. Для протоколу перевіряється також $nA \le nB$. Усі співвідношення перевірено assert.

3. Які дані потрібні процедурам RSA?

Encrypt(M, відкритий ключ адресата n,e); Decrypt(C, секретні d,p,q); Sign(M, секретні d,p,q); Verify(M,S, відкритий ключ підписувача n,e). Не потрібно передавати всі параметри, оскільки кожна операція використовує лише власну частину ключа.

4. Як забезпечується конфіденційність і справжність?

A підписує k своїм секретним ключем, після чого шифрує k і підпис відкритим ключем B. Лише B може відновити їх своїм секретним ключем, а підпис перевіряється відкритим ключем A.

5. Що перевіряє одержувач?

Коректність діапазону отриманого k ($0<k<nA$), умову $nA \le nB$ і рівність $k=S^{eA} \bmod nA$. Також потрібно попередньо довірено встановити належність відкритого ключа A

відповідному абоненту; інакше математично правильний підпис сам собою не засвідчує реальної особи.

Висновки

У ході комп’ютерного практикуму реалізовано генерацію 256-бітних простих чисел, побудовано RSA-ключі для двох абонентів, перевірено коректність шифрування, розшифрування, цифрового підпису й протоколу конфіденційного передавання ключа. Усі локальні перевірки пройдено. Під час тестування із зовнішнім середовищем підтверджено п’ять успішних перевірок: шифрування і розшифрування в обох напрямках, цифрові підписи в обох напрямках і передавання ключа від Python до сервера. У зворотному протоколі Server SendKey → Python ReceiveKey отримано негативний результат, який задокументовано для подальшого виправлення. Навчальна реалізація RSA призначена лише для демонстрації алгоритмів.